

Rapport de Cadrage de Projet : Architecture Zero Trust avec IAM/PIM

1. Titre du projet

- Architecture Zero Trust : Réduction de la surface d’attaque par une gestion d’identités et de privilèges basée sur le modèle IAM/PIM.

2. Alternatives architecturales (Cloud vs On-Premise)

Le choix de la plateforme de gestion des identités et des accès (IAM) est un pilier fondamental de toute stratégie Zero Trust. Deux approches principales s'opposent :

- Solution Cloud (SaaS - Software as a Service) :** Des plateformes comme Microsoft Entra ID (Azure AD), Okta ou Duo offrent des services IAM complets, managés et hautement disponibles. L'enjeu principal réside dans l'externalisation de la gestion de l'identité, impliquant une dépendance à un fournisseur tiers. Bien que le déploiement soit accéléré et la charge opérationnelle réduite, cela pose des questions de souveraineté des données, de flexibilité (enfermement propriétaire ou *vendor lock-in*) et de coûts récurrents (modèle par abonnement). L'interopérabilité est excellente mais souvent contrainte à l'écosystème du fournisseur.
- Solution On-Premise / Auto-hébergée :** Des solutions Open Source comme Authentik, Keycloak ou FreeIPA permettent de construire une infrastructure IAM/PIM sur ses propres serveurs (physiques ou virtualisés) ou sur une infrastructure IaaS (Infrastructure as a Service) privée. Cette approche garantit une **souveraineté totale des données** et une flexibilité maximale pour l'intégration et la personnalisation. Cependant, elle induit une complexité de déploiement et de maintenance significativement plus élevée, nécessitant des compétences internes en administration système, en sécurité et en gestion de bases de données. La haute disponibilité et la scalabilité deviennent la responsabilité de l'équipe projet.

3. Étude comparative des solutions

Le tableau suivant synthétise les différences majeures entre les deux modèles.

Critère	Solution Commerciale (SaaS - ex: Entra ID)	Solution Open Source (Auto-hébergée - ex: Authentik)
Coût (TCO)	Coûts d'abonnement récurrents (OPEX), prévisibles mais potentiellement élevés à grande échelle.	Coût initial faible (licences gratuites), mais coûts cachés importants (CAPEX/OPEX) : matériel, maintenance, expertise humaine.
Flexibilité	Modérée. Les fonctionnalités sont définies par le fournisseur. Personnalisation limitée aux options de l'API et de la console.	Très élevée. Le code source est accessible. Intégrations sur-mesure et adaptations profondes sont possibles.

Critère	Solution Commerciale (SaaS - ex: Entra ID)	Solution Open Source (Auto-hébergée - ex: Authentik)
Complexité déploiement	Faible. Configuration via des interfaces graphiques assistées. Pas de gestion d'infrastructure sous-jacente.	Élevée. Nécessite le provisionnement de serveurs, la configuration de bases de données, réseaux, et la sécurisation de l'ensemble.
Souveraineté des données	Limitée. Les données d'identité et les métadonnées de connexion sont stockées et traitées par un tiers.	Totale. L'organisation contrôle entièrement le stockage physique et logique des données sensibles.
Support	Support professionnel garanti par le fournisseur, avec des SLA (Service Level Agreements) définis.	Support communautaire. Le support professionnel est souvent une option payante via des intégrateurs tiers.

4. La solution retenue : Authentik (IAM) + Tailscale ACL + moteur JIT Python

Suite aux contraintes du plan gratuit (SSO non couvert), la brique HashiCorp Boundary a été retirée du périmètre. La solution implémentée repose désormais sur :

- **Authentik** pour l'authentification et la gouvernance des identités (IAM).
- **Tailscale ACL** pour l'application réseau des droits d'accès.
- **Un moteur JIT développé en Python** pour l'orchestration des demandes, l'approbation, la révocation et l'expiration des accès.

1. Rôle de chaque composant

- **Authentik (IAM) :**
 - Vérifie l'identité utilisateur (SSO, groupes, permissions).
 - Fournit les claims nécessaires à l'application web (groupes, email, permissions).
- **Application Flask + DB PostgreSQL :**
 - Gère le cycle de vie des demandes d'accès (**pending**, **approved**, **denied**, **expired**).
 - Applique les règles métier :
 - blocage d'une nouvelle demande si une demande **pending** existe déjà,
 - accès autorisé si tag commun user/machine ou ACL approuvée active,
 - affichage du temps restant avant expiration des droits.
- **Moteur ACL Python (**ACL_work**) :**
 - **approve_access_request(...)** : applique la règle ACL et met à jour la DB.
 - **revoke_access_request(...)** : retire la règle ACL et met à jour la DB.
 - **cleanup_expired_requests(...)** : supprime les droits expirés et passe les statuts en **expired**.
- **Cron de maintenance :**

- Exécute périodiquement le script d'expiration pour garantir le caractère temporaire des accès JIT.

2. Flux Zero Trust retenu

1. L'utilisateur s'authentifie via Authentik.
2. Il demande un accès à une machine depuis l'UI.
3. L'application vérifie en DB l'absence de demande **pending** active.
4. Un administrateur approuve la demande.
5. Le moteur Python applique la règle ACL Tailscale pour la machine cible.
6. La demande devient **approved** avec une date d'expiration.
7. Le cron retire automatiquement les droits à expiration et bascule le statut en **expired**.

5. Étapes de développement (Roadmap)

Le plan de réalisation du PoC se décompose comme suit :

1. Phase 1 - Audit et Conception :

- Définir le périmètre du laboratoire : 1-2 applications web (ex: Grafana) et 1-2 cibles d'infrastructure (ex: un serveur SSH, une base de données PostgreSQL).
- Cartographier les flux d'accès et rédiger les politiques de sécurité cibles (ex: "Seuls les administrateurs peuvent accéder au serveur SSH de production, après validation MFA et pour une durée de 1h maximum").

2. Phase 2 - Déploiement du socle IAM (Authentik) :

- Mise en place de l'environnement de virtualisation (Docker).
- Déploiement et configuration de base d'Authentik (utilisateurs, groupes).
- Intégration d'une application web avec Authentik (via Outpost) pour valider le SSO.

3. Phase 3 - Déploiement Tailscale + ACL :

- Configuration du tailnet et des tags machines.
- Validation des API Tailscale (**TAILSCALE_API_TOKEN**, **TAILSCALE_TAILNET**).
- Vérification des règles ACL de base et de leur lisibilité.

4. Phase 4 - Implémentation du moteur JIT Python :

- Développement des fonctions d'approbation/révocation/cleanup dans **ACL_work**.
- Intégration dans les routes Flask d'administration.
- Synchronisation statuts DB <-> ACL réseau.

5. Phase 5 - Intégration UI et gouvernance des demandes :

- Blocage du bouton de demande si une requête **pending** existe.
- Affichage des accès **approved** avec timer côté utilisateur.
- Contrôle métier : accès autorisé par tag commun ou ACL approuvée active.

6. Phase 6 - Exploitation et audit :

- Mise en place du cron d'expiration.
- Vérification de la traçabilité complète (qui a demandé, qui a approuvé, quand l'accès a expiré).

6. Fonctionnalités clés à implémenter (Use Cases)

Pour valider la pertinence de la solution, les cas d'usage suivants devront être développés :

- **UC-01 : SSO et MFA pour Application Web** : Un utilisateur accède à un dashboard Grafana. Il est redirigé vers Authentik, doit s'authentifier et valider un second facteur (MFA). Authentik confirme son identité et l'autorise à accéder à Grafana.
- **UC-02 : Accès Conditionnel basé sur le Contexte (IAM)** : Le même accès à Grafana est tenté depuis une adresse IP non approuvée. Authentik bloque la tentative d'authentification en amont, même si le mot de passe est correct.
- **UC-03 : Demande d'accès JIT à une machine Tailscale** : Un utilisateur non autorisé sur une machine crée une demande d'accès. L'application enregistre la demande en **pending** et empêche les doublons.
- **UC-04 : Approbation admin et activation ACL temporaire** : Un administrateur approuve la demande, le moteur Python applique la règle ACL Tailscale, la demande passe en **approved** avec expiration.
- **UC-05 : Expiration automatique et audit** : Le cron supprime les droits à échéance, marque la demande en **expired**, et les journaux permettent de tracer tout le cycle de vie de l'accès.

1.2 Problème de sécurité identifié

Au cours de mon analyse de l'environnement VIC, j'ai constaté un problème fondamental qui affecte directement l'efficacité et la sécurité des six tâches décrites ci-dessus :

l'absence d'un contrôle centralisé, granulaire et traçable des accès privilégiés aux systèmes de l'infrastructure.

Concrètement, cette lacune se manifeste de plusieurs façons dans le quotidien de l'équipe :

1. Les analystes disposent d'accès SSH permanents sur les serveurs qu'ils doivent corriger. Ces accès ne sont pas limités dans le temps, ne sont pas journalisés de manière exploitable, et ne distinguent pas les droits selon le niveau de criticité du serveur.
2. La surface d'attaque est difficile à quantifier : il n'existe pas de vue unifiée liant les vulnérabilités détectées (via les outils de scan) aux identités des personnes ayant accès aux serveurs concernés.
3. Les accès sur les systèmes obsolètes ne sont pas contrôlés différemment des accès sur les systèmes courants, alors même que ces systèmes présentent des risques plus élevés.
□ En l'absence d'une architecture IAM/PAM/PIM, tout incident impliquant un accès privilégié (compromission de compte, action malveillante interne, erreur humaine) est très difficile à investiguer, voire impossible à prévenir proactivement.

Dans ce contexte, ce projet propose la conception d'une architecture Zero Trust intégrant IAM, PIM et PAM afin de sécuriser les accès aux systèmes et réduire la surface d'attaque dans l'environnement VIC.

1.3 Objectif de mon intervention

Mon sujet de PFE consiste à concevoir, déployer et valider une architecture Zero Trust intégrant trois composantes complémentaires : la gestion des identités et des accès (IAM), la gestion des accès privilégiés (PAM), et la gestion des identités privilégiées (PIM) avec mécanisme Just-in-Time (JIT).

Intégrer directement au périmètre du projet VIC.

□ Passer d'une situation où 'tout le monde a accès à tout à une situation où personne n'a d'accès permanent chaque accès est demandé, justifié, approuvé, limité dans le temps et enregistré

2. Principes Zero Trust appliqués au contexte VIC

2.1 Le modèle Zero Trust

Le modèle Zero Trust, formalisé par le NIST dans la publication SP 800-207, repose sur le principe fondamental « Never Trust, Always Verify » ne jamais faire confiance implicitement, toujours vérifier.

Contrairement aux architectures périmétriques traditionnelles où tout ce qui est « à l'intérieur du réseau » est considéré comme de confiance, le Zero Trust traite chaque demande d'accès comme potentiellement non fiable, qu'elle provienne de l'intérieur ou de l'extérieur du périmètre.

Dans le contexte du projet VIC, cette approche est particulièrement pertinente pour trois raisons que j'ai pu identifier en observant les pratiques en place :

- Les serveurs vulnérables (détectés par les scans CVE) sont accessibles par des comptes dont les droits ne sont pas proportionnels à la tâche à accomplir — un analyste qui doit simplement lire les logs d'un serveur dispose souvent des mêmes accès que quelqu'un qui doit le patcher.
- L'équipe travaille sur des environnements hétérogènes (serveurs Linux, Windows, bases de données, applications web) sans passerelle d'accès unifiée, ce qui rend la traçabilité quasi impossible.
- Les outils de conformité Archer nécessitent des preuves d'audit que les pratiques actuelles ne permettent pas de produire facilement.

2.2 Les trois piliers de ma solution

Ma solution s'articule autour de trois couches techniques qui correspondent directement aux trois principes Zero Trust :

Couche Outil Question Zero Trust Rôle dans le projet VIC

IAM — Identity and Access Management Authentik Qui êtes-vous ? Centraliser les identités, imposer le MFA, fédérer les authentications via OIDC

PAM — Privileged Access Management Tailscale + wazuh Que pouvez-vous faire ? Contrôler granularité des accès SSH/RDP,

RBAC, enregistrer les sessions

PIM — Privileged

Identity

— Management (JIT) Développer selon le besoin Combien de temps

? Aucun accès permanent, tout accès élevé est temporaire, approuvé et révoqué automatiquement

3. Architecture technique détaillée

3.1 Vue d'ensemble de l'architecture proposée

L'architecture que j'ai conçue pour répondre aux besoins du projet VIC s'organise en six couches fonctionnelles distinctes. Chaque couche répond à une problématique précise identifiée dans l'analyse initiale et s'appuie sur des outils open source dont j'ai vérifié la maturité et la compatibilité.

Couche Outil (Licence) Fonctionnalités clés

Couche 0 — Acteurs Équipe VIC Analystes SOC, admins réseau, ingénieurs systèmes, auditeurs conformité

Couche 1 — IAM Authentik (MIT) Portail SSO unifié, MFA TOTP/WebAuthn, groupes, LDAP server, flows d'authentification

Couche 2 — PAM Tailscale RBAC Least Privilege, Session Recording SSH/RDP/DB, Command filtering,

Audit logs JSON

Couche Outil (Licence) Fonctionnalités clés

Couche 3 —

PIM/JIT Développer selon le besoin en python Apply Asset Permission, workflow approbation, TTL configurable, révocation automatique

Couche 4 —

Vulnerability OpenVAS + DefectDojo Scan CVE/CVSS, lifecycle vulnérabilités, intégration

OTRS/BMC/Archer

Couche 5 — VPN réseau Tailscale (WireGuard) Chiffrement tunnel réseau Zero Trust, accès distant sécurisé, ACLs réseau

Couche 6 —

SIEM/Monitoring Wazuh

Ingestion logs, alertes temps réel, dashboards corrélation CVE/accès

Phase 1 : Authentification et MFA (IAM)

1. L'utilisateur tente d'accéder à une application ou un service via le Portail (par exemple Flexera, OTRS ou Archer).
2. Il est ensuite redirigé vers le système Authentik, où il saisit ses identifiants (login et mot de passe).
3. Authentik émet un ID Token (SSO) qui doit être validé par le système pour garantir l'authenticité de l'utilisateur.
4. L'utilisateur doit ensuite passer par la vérification du second facteur MFA via un code TOTP, assurant une double sécurité avant de poursuivre.

Phase 2 : Gestion des accès (PAM / RBAC / Least Privilege)

5. Les utilisateurs validés sont synchronisés via OIDC Provider d'Authentik avec Tailscale.
6. Tailscale applique les politiques RBAC et le principe de Least Privilege, en utilisant les ACLs pour contrôler l'accès aux ressources.
7. Les rôles et permissions sont vérifiés automatiquement :
 - ☐ Si l'accès est autorisé, l'utilisateur peut continuer.
 - ☐ Si l'accès est refusé, l'utilisateur est redirigé vers une page Access Denied.

Phase 3 : Just-in-Time Access (PIM) et suivi des sessions

8. Après autorisation, l'utilisateur peut créer une demande d'accès JIT pour accéder à une ressource critique, par exemple pour corriger une vulnérabilité spécifique (CVE 2025-XXXX).
9. Le système approuve la demande avec un TTL limité (durée d'accès définie).
10. L'utilisateur peut accéder à la ressource uniquement pendant la période autorisée. La session (ex. SSH Tunnel) reste active et toutes les actions sont enregistrées.
11. Les logs de session sont collectés et centralisés via Wazuh SIEM pour le suivi et l'audit.
12. À l'expiration du TTL, une révocation automatique est effectuée : la session est fermée immédiatement et l'accès est révoqué.

3.2 Description détaillée de chaque composant

3.2.1 Authentik — Couche IAM

Authentik est le composant central de gestion des identités, il joue le rôle de fournisseur d'identité (IdP) pour l'ensemble de l'architecture.

Dans le contexte de notre projet Authentik est dédiée pour répondre à plusieurs besoins spécifiques de l'équipe VIC :

- Annuaire central : tous les comptes des membres de l'équipe VIC sont créés dans Authentik et

organisés en groupes correspondant aux niveaux d'accès (admins, analystes, auditeurs, demandeurs JIT).

- MFA obligatoire : le flow d'authentification impose le TOTP (Google Authenticator / Authy) à chaque connexion, sans exception. Un utilisateur sans MFA configuré est bloqué jusqu'à ce qu'il le configure c'est le mécanisme «Force Configure» .
- Serveur LDAP intégré : Authentik expose un serveur LDAP (port 389) que JumpServer consomme pour synchroniser les utilisateurs et groupes automatiquement. Cela évite la double saisie et garantit que la révocation dans Authentik se propage immédiatement dans JumpServer.
- Serveur OIDC (OpenID Connect) intégré : Tailscale peut utiliser ces informations pour appliquer des ACLs (Access Control Lists) et définir qui peut se connecter à quelles machines ou services.
- SSO pour les outils web : Authentik est également le portail SSO pour Grafana, DefectDojo et le panel d'administration Wazuh, via le protocole OIDC

3.2.2 Tailscale : Couches PAM et PIM

JumpServer assure simultanément les fonctions PAM (contrôle et enregistrement des sessions) et PIM (gestion des accès Just-in-Time).

J'ai choisi Tailscale après avoir évalué plusieurs alternatives (Teleport CE, HashiCorp Boundary). La raison principale est que Tailscale intègre nativement le RBAC, le JIT le session recording et l'authentification OIDC sans restriction sur le nombre d'utilisateurs ni de sessions.

Configuration du RBAC Least Privilege que j'ai mise en place :

- Chaque actif (serveur) est étiqueté avec des labels : env=dev, env=staging, env=prod, criticality=low/medium/high/.
- Les permissions sont définies par groupe et par label : un analyste du groupe jumpserverdevops ne voit que les assets avec label env=dev ou env=staging.
- Les logins root et les commandes dangereuses (rm -rf, shutdown, etc.) sont filtrés et bloqués par les ACL Tailscale.
- Chaque session SSH/RDP est enregistrée intégralement

Accès Just-in-Time (JIT) via Tailscale

Tailscale permet de sécuriser l'accès Just-in-Time aux ressources critiques en créant des tunnels réseau privés éphémères basés sur les identités des utilisateurs et les politiques d'accès.

1. Aucun accès par défaut :

Les utilisateurs n'ont pas de connexion permanente aux serveurs sensibles. L'accès est strictement contrôlé et temporaire.

2. Création automatique de tunnels sécurisés :

Lorsqu'un utilisateur est autorisé à accéder à une machine, Tailscale établit un tunnel réseau chiffré directement entre le client de l'utilisateur et le serveur cible, sans exposer ce dernier sur Internet.

3. Contrôle d'accès basé sur les rôles et ACL :

Tailscale applique les ACL (Access Control Lists) et le RBAC, garantissant que l'utilisateur n'accède qu'aux ressources pour lesquelles il a été autorisé et uniquement pendant la période définie.

4. Durée limitée et révocation automatique :

L'accès est limité dans le temps (TTL) : à l'expiration, le tunnel est automatiquement fermé et l'accès est révoqué, assurant le respect du principe du least privilege.

aucun accès permanent sur les serveurs de production. Chaque accès est justifié, approuvé par un pair, limité dans le temps, et laisse une trace auditée. C'est exactement ce dont l'équipe VIC a

besoin pour satisfaire les exigences de conformité Archer.

3.2.3 Tailscale — Couche VPN Zero Trust réseau

En complément de la couche PAM/PIM, j'ai intégré Tailscale comme solution de VPN Zero Trust basé sur le protocole WireGuard. Contrairement aux VPNs traditionnels qui accordent un accès réseau global une fois authentifié, Tailscale applique des ACLs granulaires au niveau réseau on ne peut accéder qu'aux ressources explicitement autorisées.

Dans le contexte du projet VIC, Tailscale joue un rôle complémentaire :

- Chiffrement de bout en bout : tous les flux réseau entre les postes des analystes et les serveurs VIC passent par un tunnel WireGuard chiffré, même sur le réseau interne de l'entreprise.
- ACLs réseau Zero Trust : un analyste connecté à Tailscale n'a accès qu'aux serveurs explicitement autorisés dans la politique ACL Tailscale une couche de contrôle réseau en plus du contrôle applicatif JumpServer.
- Accès distant sécurisé : pour les membres de l'équipe qui travaillent en télétravail ou sur site client, Tailscale remplace avantageusement un VPN d'entreprise classique avec une sécurité équivalente mais une gestion plus simple.
- Intégration SSO : Tailscale peut être configuré pour s'authentifier via Authentik (OIDC), ce qui unifie la gestion des identités pour la couche réseau également.

3.2.4 OpenVAS + DefectDojo Couche Vulnerability

Ces deux outils constituent le cœur du projet VIC et existaient déjà dans l'environnement de l'équipe. Mon travail a consisté à les intégrer dans l'architecture Zero Trust de façon à créer une corrélation entre les vulnérabilités détectées et les accès contrôlés.

- OpenVAS : scanne les mêmes assets que ceux déclarés dans JumpServer. Les résultats sont importés dans DefectDojo via son API REST.
- DefectDojo gère le cycle de vie complet : création automatique de tickets OTRS/BMC lors de la détection d'une CVE critique, suivi de la remédiation, fermeture automatique après vérification post-patch.

3.2.5 Wazuh Couche SIEM et Monitoring

Cette dernière couche assure la visibilité et la corrélation en temps réel de tous les événements de sécurité générés par les couches supérieures.

- Wazuh ingère les logs JSON de JumpServer (sessions, tickets JIT, commandes filtrées) et les logs d'Authentik (authentifications, échecs MFA, révocations). Il génère des alertes en temps réel sur les comportements anormaux.

4. Contribution de l'architecture aux tâches du projet VIC

Aspect Détail

Problème avant Les analystes accèdent aux serveurs vulnérables sans qu'on sache exactement qui a touché à quoi. La priorisation des corrections n'est pas liée à la criticité de l'accès.

Contribution Zero Trust JumpServer et Tailscale labellise les assets avec leur niveau de criticité (criticality=high/critical). OpenVAS scanne ces mêmes assets. DefectDojo croise le score CVSS avec le label JumpServer pour prioriser automatiquement : un CVE 9.0 sur un asset criticality=critical génère automatiquement un ticket OTRS P1.

Outil principal OpenVAS + DefectDojo + JumpServer Assets + Grafana Dashboard

Aspect Détail

Problème avant L'évaluation du risque ne prenait pas en compte qui avait accès aux systèmes vulnérables.

Contribution Zero Trust Authentik fournit la liste des identités ayant accès à chaque serveur (via les groupes JumpServer). DefectDojo calcule un score de risque contextualisé : CVE critique + accès non contrôlé = risque très élevé VS CVE critique + accès JIT traçable = risque modéré. Cette contextualisation est exportée vers Archer.

Outil principal Authentik + DefectDojo + Archer GRC

Valeur ajoutée La qualification du risque passe de ce serveur a une CVE critique à 'ce serveur a une CVE critique ET X personnes y ont un accès permanent non justifié'.

Cycle de vie des vulnérabilités

C'est la tâche pour laquelle ma solution apporte la valeur la plus importante. Le cycle complet détection → remédiation → vérification est désormais entièrement traçable :

1. Détection : OpenVAS détecte CVE-2024-XXXX sur le serveur prod-db-01 (label criticality=high).
2. Remédiation : l'analyste crée un ticket JIT dans JumpServer ou Tailscale avec la raison «Correction CVE-2024-XXXX — OTRS #4521». L'admin approuve pour 2 heures.
3. La session de correction est enregistrée intégralement par JumpServer et Tailscale.
4. Vérification : nouveau scan OpenVAS post-correction. Si la CVE est corrigée, DefectDojo ferme le ticket avec référence à la session JumpServer et Tailscale comme preuve.
5. Tout l'audit trail est exporté vers Archer pour la conformité.
6. Métriques de réduction de la surface d'attaque
La valeur d'une architecture de sécurité ne peut être appréciée qu'à travers des métriques concrètes. J'ai défini les indicateurs suivants pour quantifier l'apport de ma solution par rapport à la situation initiale :

Métrique Avant déploiement Après déploiement Gain estimé

Comptes SSH permanents sur serveurs de prod N (non inventoriés) 0 (tout passe par JIT) Élimination 100%

Durée moyenne des accès

privilégiés Permanente (sans

fin) 1h à 4h (TTL JIT) Réduction ≥ 98%

Sessions enregistrées et auditable 0% des sessions 100% via

JumpServer/Tailscale Couverture totale

Métrique Avant déploiement Après déploiement Gain estimé

MFA imposé sur accès aux systèmes 0% (aucun contrôle) 100% via Authentik Couverture totale

Temps de détection d'accès anormal Manuel ,plusieurs jours < 5 minutes (Wazuh) Amélioration ≥ 99%

CVEs sans accès contrôlé associé Tous les serveurs 0 (RBAC

JumpServer/Tailscale) Réduction 100%

Traçabilité pour Archer/OTRS Difficile à obtenir Log JSON + session replay De 0% à 100%

Révocation accès lors départ employé Manuelle plusieurs jours Immédiate via Authentik Délai 0

Conclusion et Perspectives

Ce projet de PFE m'a permis de concevoir une architecture Zero Trust complète en réponse à un problème réel identifié au sein de l'équipe VIC. En combinant Authentik (IAM), JumpServer CE, Tailscale (PAM + PIM) et les outils de monitoring .

Les points clés que je retiens de ce projet sont les suivants :

- Le Zero Trust n'est pas un produit qu'on achète, c'est une démarche qu'on construit. Chaque couche de l'architecture répond à une question précise : Qui es-tu ? Que peux-tu faire ? Combien de temps ? Et la réponse à ces trois questions ensemble constitue le fondement d'une posture de sécurité solide.
- L'intégration avec le contexte métier est essentielle. Ma solution ne serait pas pertinente si elle ne répondait pas directement aux six tâches du projet VIC. C'est cette intégration qui lui donne sa valeur réelle.
- L'open source est une alternative viable et mature pour des projets de sécurité d'entreprise. Authentik et JumpServer CE/Tailscale offrent des fonctionnalités comparables à des solutions commerciales coûteuses.